

Восстановление графа по матрице наблюдений

Описание реализованного алгоритма · MNIST · версия для разбора кода

Задача состоит в выборе соседей каждого признака по множеству наблюдений. В MNIST наблюдение — изображение, признак — яркость пикселя. Один и тот же неизвестный порядок пикселей используется во всех изображениях. Алгоритм строит общий неориентированный граф: ребро разрешает использовать одну вершину для линейного предсказания другой. Результат представляет разреженный граф предсказательных зависимостей между признаками.

1 Последовательность действий

1. Разделить изображения на обучающую и проверочную части для восстановления структуры.
2. По обучающей части выделить активные признаки и стандартизовать их значения.
3. Вычислить корреляции всех активных пар; для каждой вершины оставить K кандидатов.
4. Построить локальные Lasso-модели, выбрать их регуляризацию по BIC и получить варианты стартового графа.
5. Оценить варианты через Ridge-предсказание на проверочной части и штраф за рёбра.
6. Удалить рёбра, удаление которых уменьшает энергию.
7. Изменять граф стохастическими добавлениями и удалениями, сохраняя лучшее найденное состояние.
8. Вернуться к лучшему состоянию, повторить удаление вредных рёбер и вернуть граф.



2 Данные и обозначения

Вход: $X^{\text{tr}} \in \mathbb{R}^{n \times N}$ и $X^{\text{val}} \in \mathbb{R}^{m \times N}$. Строки — наблюдения, столбцы — одни и те же вершины. Граф $G = (V, E)$ содержит N вершин; $V_a \subseteq V$ — активное множество, $p = |V_a|$, $\mathcal{N}_G(i)$ — соседи вершины i . Рёбра неориентированные, без петель. Неактивные вершины возвращаются изолированными.

Алгоритм получает только эти две матрицы и параметры. Координаты, метки цифр, эталонные рёбра и их количество не передаются. Входное API: `recover_graph(X_train, X_val, config)` в `src/latent_graph.py`.

3 Активные вершины и нормировка

На обучающей части вычисляются среднее μ_i , стандартное отклонение σ_i и частота заметных значений f_i :

$$f_i = \frac{1}{n} \sum_{s=1}^n \mathbf{1}\{|X_{si}^{\text{tr}}| > \tau_x\}, \quad V_a = \{i : \sigma_i > \tau_\sigma, f_i > \tau_f\}. \quad (1)$$

Имя `variance_threshold` в коде историческое: порог сравнивается со *стандартным отклонением*, а не с дисперсией. Для MNIST после деления яркости на 255 используются $\tau_\sigma = 0.02$, $\tau_x = 0.05$, $\tau_f = 0.01$. Для $i \in V_a$ обе части нормируются одними обучающими статистиками:

$$Z_{si}^{\text{tr}} = \frac{X_{si}^{\text{tr}} - \mu_i}{\sigma_i}, \quad Z_{si}^{\text{val}} = \frac{X_{si}^{\text{val}} - \mu_i}{\sigma_i}. \quad (2)$$

Почти постоянный признак не даёт достаточной вариативности для надёжного выбора соседей. Фильтрация оставляет вершины с достаточной вариативностью наблюдений.

4 Все пары и список кандидатов

На всей обучающей части вычисляется корреляция ρ_{ij} . Её строки также случайно делятся на $B = 4$ блока, и в каждом вычисляется $\rho_{ij}^{(b)}$. Все блоки содержат те же столбцы. Оценка пары равна

$$s_{ij} = \frac{|\rho_{ij}| \left| B^{-1} \sum_{b=1}^B \text{sign}(\rho_{ij}^{(b)}) \right|}{1 + \text{Std}_b(|\rho_{ij}^{(b)}|)}. \quad (3)$$

Согласованность знаков повышает оценку, изменчивость модуля её понижает. В текущем исходнике неопределённая корреляция постоянного столбца внутри блока заменяется нулём. Эта защитная обработка была добавлена после исторических MNIST-прогнозов.

Для каждой вершины i множество C_i содержит $K = 24$ других вершины с наибольшими s_{ij} . Объединение неориентированных пар (i, j) , $j \in C_i$, образует список предложений P . Он ещё не является графом. K не ограничивает итоговую степень: вершину могут выбрать другие вершины, а последующий поиск допускает пары вне P .

На этом этапе обрабатываются все $p(p-1)/2$ пары. Кроме того, код строит полные матрицы Грама для оценки энергии. Исчерпывающего перебора *всех графов* или всех условных соседств нет.

5 Lasso, BIC и начальный граф

Для каждой вершины обучаются пять локальных моделей:

$$\hat{\beta}_i(a) = \arg \min_{\beta} \left\{ \frac{1}{2n} \|Z_i^{\text{tr}} - Z_{C_i}^{\text{tr}} \beta\|_2^2 + a \|\beta\|_1 \right\}, \quad a \in \{0.003, 0.006, 0.012, 0.025, 0.05\}. \quad (4)$$

Lasso зануляет часть коэффициентов. Из этой конечной сетки выбирается минимум

$$\text{BIC}_i(a) = n \log \left(\frac{\text{RSS}_i(a)}{n} \right) + k_i(a) \log n, \quad (5)$$

где RSS — сумма квадратов ошибок самой Lasso-модели на train, ограниченная снизу числом 10^{-12} , а k_i — число коэффициентов с модулем больше 10^{-8} . Отдельного OLS-переобучения выбранной поддержки нет; это применённая в коде BIC-подобная эвристика отбора.

Пусть D — направленные предложения: $(i, j) \in D$, если коэффициент j в модели i ненулевой. Проверяются три старта:

$$E_{\cup} = \{(i, j) : (i, j) \in D \text{ или } (j, i) \in D\}, \quad (6)$$

$$E_{\cap} = \{(i, j) : (i, j) \in D \text{ и } (j, i) \in D\}, \quad (7)$$

$$G_0 = \arg \min_{G: E(G) \in \{\emptyset, E_{\cup}, E_{\cap}\}} \mathcal{E}(G). \quad (8)$$

Здесь $E(G)$ обозначает множество рёбер, а $\mathcal{E}(G)$ — энергию из следующего раздела. Отжиг начинается с варианта, выбранного по минимальной энергии среди трёх стартов.

6 Ridge-предсказание и энергия графа

Для заданного соседства $Q = \mathcal{N}_G(i)$ коэффициенты оцениваются на train:

$$\hat{w}_i(Q) = \arg \min_w \left\{ \frac{1}{n} \|Z_i^{\text{tr}} - Z_Q^{\text{tr}} w\|_2^2 + \alpha \|w\|_2^2 \right\}. \quad (9)$$

Ridge нужен для устойчивого предсказания при коррелированных соседях. Он оценивает уже выбранное соседство, а не выбирает рёбра. При $\alpha = 0.01$ решение и проверочная ошибка вычисляются через

$$C = \frac{(Z^{\text{tr}})^\top Z^{\text{tr}}}{n}, \quad H = \frac{(Z^{\text{val}})^\top Z^{\text{val}}}{m}, \quad (10)$$

$$\hat{w}_i = (C_{QQ} + \alpha I)^{-1} C_{Qi}, \quad (11)$$

$$v_i(G) = H_{ii} - 2\hat{w}_i^\top H_{Qi} + \hat{w}_i^\top H_{QQ} \hat{w}_i. \quad (12)$$

В коде система решается через `pr.linalg.solve`, без явного обращения матрицы. При $Q = \emptyset$ предсказание стандартизованного признака равно нулю и $v_i = H_{ii}$.

$$\mathcal{E}_\lambda(G) = \sum_{i \in V_a} \log \max\{v_i(G), 10^{-12}\} + 2\lambda|E|. \quad (13)$$

Каждое ребро входит в степени двух концов, поэтому его цена равна 2λ . При $\lambda = 0.08$ цена — 0.16. Внутренний Ridge-штраф $\alpha\|w\|^2$ не прибавляется к энергии: он влияет на коэффициенты и тем самым на проверочную MSE. Логарифм оценивает относительное изменение ошибки. Отрицательная энергия допустима; выбирается меньшее значение. Ни предсказания GNN, ни её loss здесь не используются.

7 Pruning и точное изменение энергии

Для переключения пары $e = \{i, j\}$ строится $G \triangle e$: существующее ребро удаляется, отсутствующее добавляется. Вычисляется

$$\Delta_e(G) = \mathcal{E}_\lambda(G \triangle e) - \mathcal{E}_\lambda(G). \quad (14)$$

Меняются только соседства i и j , поэтому достаточно заново решить две Ridge-задачи.

Начальный pruning проходит существующие рёбра в сортированном порядке. Если удаление даёт $\Delta_e < -10^{-10}$, оно выполняется немедленно. Проход повторяется не более шести раз; остановка наступает раньше, если ничего не удалено.

8 Отжиг: предложения, история и принятие

На каждом из $S = 2000$ шагов формируются 12 предложений. При непустом графе ветвь DELETE выбирается с вероятностью 0.55, ребро — равномерно из существующих. В другой ветви 10% предложений выбираются из всех активных пар, остальные — из P . Это условные 10% ADD-ветви, приблизительно 0.045 всех предложений при непустом графе. Поиск отсутствующей пары делает до 20 попыток; при неудаче код всё равно переключает последнюю пару. Поэтому фактический тип хода определяется её наличием в графе.

Для всех предложений вычисляется актуальное Δ_e . История хранит три ЕМА:

$$m_e \leftarrow \beta m_e + (1 - \beta) \Delta_e, \quad q_e \leftarrow \beta q_e + (1 - \beta) \Delta_e^2, \quad (15)$$

$$z_e \leftarrow \beta z_e + (1 - \beta) \text{sign}(\Delta_e), \quad \beta = 0.9. \quad (16)$$

Обновляется история только *выбранного* предложения, даже если ход отклонён. При первом рассмотрении используются $(\Delta_e, \Delta_e^2, \text{sign } \Delta_e)$. Ранг предложения на шаге $t = 0, \dots, S - 1$:

$$u_e = \sqrt{\max(q_e - m_e^2, 0)}, \quad \eta_t = \text{clip}\left(\frac{t/S - 0.10}{0.20}, 0, 1\right), \quad (17)$$

$$R_e(t) = \Delta_e + \eta_t(0.15u_e - 0.10|z_e|). \quad (18)$$

Выбирается минимальный ранг. Среднее m_e не добавляется к рангу напрямую; Знак истории используется по модулю, поощряя устойчивость направления изменения энергии. Для ADD и DELETE используется общая история пары.

Начальная температура калибруется по модулям изменений до 64 пар из P :

$$T_0 = \max\{\text{median}(|\Delta_e|), 10^{-3}\}, \quad T_f = 10^{-3}T_0, \quad (19)$$

$$T_t = T_0(T_f/T_0)^{t/\max(S-1,1)}, \quad P_{\text{accept}} = \min\{1, \exp(-\Delta_e/T_t)\}. \quad (20)$$

Принятие зависит от текущей энергии; история влияет только на выбор пары. Сохраняется граф с наименьшей посещённой энергией. После отжига к нему применяется ещё до шести проходов pruning. Таким образом, реализован эвристический поиск с температурным принятием и выбором перспективных предложений по истории.

9 Параметры, выход и вычислительная сложность

В экспериментальном запуске `run_mnist.py`, вне функции восстановления, сравниваются $\lambda \in \{0.08, 0.10, 0.12\}$. Вычисляется непениализованный критерий

$$L_{\text{val}}(G) = \mathcal{E}_\lambda(G) - 2\lambda|E|, \quad L_{\text{val}}(G) \leq L_* + 0.01|L_*|. \quad (21)$$

Из допустимых вариантов выбирается граф с минимальным числом рёбер. Здесь L_* — минимум L_{val} среди трёх полученных графов. Это допуск по сумме логарифмов MSE, а не по MSE и не по accuracy. Одна и та же structure-validation используется для поиска графа и выбора штрафа.

Функция возвращает `adjacency` и объект модели с `edges_`, `active_mask_`, `best_energy_`, параметрами и диагностикой стадий. Оценка всех корреляций и построение Грам-матриц требуют $O((n+m)p^2)$ операций; хранение — $O((n+m)p + Bp^2)$. Текущая реализация дополнительно материализует список всех пар для глобальных предложений, поэтому большие графы требуют оптимизации памяти. Один пересчёт локальной модели при степени d имеет стоимость порядка $O(d^3)$.

Минимизация энергии выбирает соседства, которые хорошо предсказывают признаки при заданной цене рёбер. В эксперименте MNIST такой выбор позволил восстановить большую часть рёбер активной решётки; численные результаты приведены в отдельном отчёте.

Навигация по исходникам. Все этапы восстановления находятся в `LatentGraphRecovery.fit`: `local_score` реализует формулу (13), `delta` — переключение ребра. `src/run_mnist.py` задаёт разбиения и сравнение штрафов. `src/check_package.py` проверяет формулу Ridge-ошибки и эквивалентность исходников; `src/build_assets.py` сверяет изображаемые графы с сохранёнными метриками.